

AI HW3

1. CNN

CNN uses filters to capture the features of the RGB images, below is the implementation.

- Data Loading

```
def load_train_dataset(path: str='data/train/') -> Tuple[List, List]:
    images = []
    labels = []
    label_map = {"elephant": 0, "jaguar": 1, "lion": 2, "parrot": 3, "penguin": 4}

    for animal in os.listdir(path):
        animal_path = os.path.join(path, animal)
        if os.path.isdir(animal_path):
            for img_file in os.listdir(animal_path):
                images.append(os.path.join(animal_path, img_file))
                labels.append(label_map[animal])
    return images, labels

def load_test_dataset(path: str='data/test/') -> List:
    images = []
    for img_file in os.listdir(path):
        images.append(os.path.join(path, img_file))
    return images
```

For training data, we need to give both X and y.

Also, in order to estimate the performance of Kaggle submissions, in main.py, it will split the train data to train and valid.

For test data, we just need to give the X.

- Design Model Architecture

```
class CNN(nn.Module):
    def __init__(self, num_classes=5):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(64 * 56 * 56, 256)
        self.fc2 = nn.Linear(256, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

I just use the simple architecture, with filter of 3*3 which I think most people will use in image recognition.

Below is the change of data shape

Input : (3, 224 , 224)

Conv1 : (32 , 224 ,224)

Pool1 : (32, 112, 112)

Cov2 : (64, 112 , 112)

Pool2 : (64 , 56 , 56)

fc1 : (256)

fc2 : (5)

- Define function train(), validate() and test()

```
def train(model: CNN, train_loader: DataLoader, criterion, optimizer, device) -> float:
    model.train()
    running_loss = 0.0
    for inputs, labels in tqdm(train_loader, desc='Training'):
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * inputs.size(0)

    avg_loss = running_loss / len(train_loader.dataset)
    return avg_loss
```

```
def validate(model: CNN, val_loader: DataLoader, criterion, device) -> Tuple[float, float]:
    model.eval()
    running_loss = 0.0
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for inputs, labels in tqdm(val_loader, desc='Validating'):
            inputs, labels = inputs.to(device), labels.to(device)

            outputs = model(inputs)
            loss = criterion(outputs, labels)

            running_loss += loss.item() * inputs.size(0)

            preds = outputs.argmax(dim=1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    avg_loss = running_loss / len(val_loader.dataset)
    accuracy = (torch.tensor(all_preds) == torch.tensor(all_labels)).float().mean().item()
    return avg_loss, accuracy
```

use the training model to get the prediction, and then get the accuracy at the validation set.

```
def test(model: CNN, test_loader: DataLoader, device):
    model.eval()
    results = []

    with torch.no_grad():
        for inputs, paths in tqdm(test_loader, desc='Testing'):
            inputs = inputs.to(device)

            outputs = model(inputs)
            preds = outputs.argmax(dim=1).cpu().numpy()

            for img_name, pred in zip(paths, preds):
                results.append({'id': img_name, 'prediction': pred})

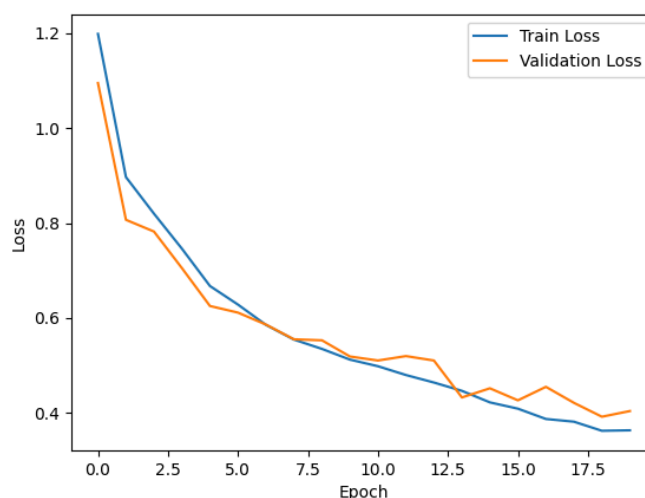
    df = pd.DataFrame(results)
    df.to_csv('CNN.csv', index=False)
    print(f"Predictions saved to 'CNN.csv'")
    return
```

save the prediction of the test data in order to submit it to Kaggle.

- Plot Training and Validation Loss

```
def plot(train_losses: List, val_losses: List):
    plt.plot(train_losses, label='Train Loss')
    plt.plot(val_losses, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.savefig('loss.png')
    plt.close()
    print("Save the plot to 'loss.png'")
```

- Experiments



This is the loss, as you can see, it has not encountered any fitting problems yet. But due to time constraints, I stopped training. As for how to reduce the chance of overfitting, both regularization and dropout can do that. I forget to take the screenshot of the accuracy change after I modify the code, but I take the screenshot of the Kaggle submissions.

✓	CNN.csv Complete · 2h ago	0.813	☐
✓	CNN.csv Complete · 3h ago	0.826	☐

As you can see, the performance didn't increase. I guess the reason is because the dataset is small.

2. Decision Tree

- Features extraction

```
def get_features_and_labels(model: ConvNet, dataloader: DataLoader, device) -> Tuple[List, List]:
    model.eval()
    features = []
    labels = []
    with torch.no_grad():
        for inputs, batch_labels in tqdm(dataloader, desc='Extracting features'):
            inputs = inputs.to(device)
            outputs = model(inputs)
            features.append(outputs.cpu().numpy())
            labels.append(batch_labels.numpy())
    features = np.vstack(features)
    labels = np.concatenate(labels)
    return pd.DataFrame(features), labels

def get_features_and_paths(model: ConvNet, dataloader: DataLoader, device) -> Tuple[List, List]:
    model.eval()
    features = []
    paths = []
    with torch.no_grad():
        for inputs, batch_paths in tqdm(dataloader, desc='Extracting test features'):
            inputs = inputs.to(device)
            outputs = model(inputs)
            features.append(outputs.cpu().numpy())
            paths.extend(batch_paths)
    features = np.vstack(features)
    return pd.DataFrame(features), paths
```

get the 300 features from ConvNet

- Model Architecture

```

def _build_tree(self, X: pd.DataFrame, y: np.ndarray, depth: int):
    if (len(np.unique(y)) == 1 or
        depth >= self.max_depth or
        len(X) < self.min_samples_split):
        return np.bincount(y).argmax()

    best_feature_index, best_threshold, best_gain = self._best_split(X, y)

    if best_feature_index is None or best_gain < self.min_impurity_decrease:
        return np.bincount(y).argmax()

    left_X, left_y, right_X, right_y = self._split_data(X, y, best_feature_index, best_threshold)

    if len(left_y) < self.min_samples_leaf or len(right_y) < self.min_samples_leaf:
        return np.bincount(y).argmax()

    self.progress.update(1)

    return {
        'feature_index': best_feature_index,
        'threshold': best_threshold,
        'left': self._build_tree(left_X, left_y, depth + 1),
        'right': self._build_tree(right_X, right_y, depth + 1)
    }

```

Use pre-pruning to reduce overfitting

Including

- max_depth
- min_sample_split
- min_sample_leaf
- min_impurity_decrease

```

def predict(self, X: pd.DataFrame) -> np.ndarray:
    predictions = [self._predict_tree(row, self.tree) for _, row in X.iterrows()]
    return np.array(predictions)

```

Use the _predict_tree to traverse, and get the classify result.

```

def _predict_tree(self, x, tree_node):
    if not isinstance(tree_node, dict):
        return tree_node

    feature_val = x[tree_node['feature_index']]

    if feature_val < tree_node['threshold']:
        return self._predict_tree(x, tree_node['left'])
    else:
        return self._predict_tree(x, tree_node['right'])

```

Check whether the tree_node is leaf, if yes, it will return the class, else, keep traversing to the left and right.

```
def _split_data(self, X: pd.DataFrame, y: np.ndarray, feature_index: int, threshold: float):
    left_mask = X.iloc[:, feature_index] < threshold
    right_mask = ~left_mask
    left_dataset_X = X[left_mask]
    left_dataset_y = y[left_mask]
    right_dataset_X = X[right_mask]
    right_dataset_y = y[right_mask]
    return left_dataset_X, left_dataset_y, right_dataset_X, right_dataset_y
```

splitting the dataset into right and left.

```
def _best_split(self, X: pd.DataFrame, y: np.ndarray):
    best_gain = -1
    best_feature_index, best_threshold = None, None

    for feature_index in range(X.shape[1]):
        thresholds = X.iloc[:, feature_index].unique()
        for threshold in thresholds:
            left_mask = X.iloc[:, feature_index] < threshold
            right_mask = ~left_mask

            if left_mask.sum() == 0 or right_mask.sum() == 0:
                continue

            gain = self._information_gain(y, y[left_mask], y[right_mask])

            if gain > best_gain:
                best_gain = gain
                best_feature_index = feature_index
                best_threshold = threshold

    return best_feature_index, best_threshold
```

Evaluate all possible splits for the current dataset and select the one that maximizes information gain or minimizes entropy.

```
def _entropy(self, y: np.ndarray) -> float:
    if len(y) == 0:
        return 0
    hist = np.bincount(y)
    ps = hist / len(y)
    return -np.sum([p * np.log2(p) for p in ps if p > 0])
```

Calculate the entropy of the target labels.

- Experiment

In this experiment, when I changed the max_depth to 5, it should have higher accuracy, since it can reduce overfitting. But it didn't, the reason is probably because in the case of max_depth == 7, it has not encountered any fitting problems yet. Therefore, when I changed the max_depth to 9, it had a higher performance.